AckerRadar – Case Study

Enterprise REST API for Precision Agriculture

Yves Kühn

2026

Contents

AckerRadar – Case Study **1**
At a Glance . . . 1
The Problem . . . 1
Requirements . . . 2
Architecture . . . 2
Technical Decisions and Trade-offs . . . 5
Sequence Diagram: Creating a Plot . . . 6
Challenges and Solutions . . . 6
Results . . . 7
Lessons Learned . . . 8
About Me . . . 8

AckerRadar – Case Study

A Multi-Tenant REST API for Precision Agriculture with Geo-Spatial Indexing, Multi-Source Data Aggregation, and ALKIS Integration

---

At a Glance

Role	Backend Developer (vocational retraining as IT Specialist for Application Development)
Company	AckerRadar UG (Berlin-based AgriTech startup)
Scope	IHK final project, part of a multi-month engagement
Stack	Java 17, Spring Boot 3.2, MongoDB (with Geo-Spatial Indexing), Maven, JWT, OpenAPI 3
Domain	Precision agriculture, geo-spatial data, sensor integration, market data aggregation
Status	IHK certification passed; codebase is proprietary (NDA)

The Problem

Agricultural businesses in Germany work with fragmented data sources: weather data comes from one system, commodity prices from another, soil sensors from a third, and official cadastral land records (ALKIS) from a fourth. The consequence: decisions about fertilization, planting, and optimal selling time are based on manually aggregated data, often delayed by hours or days.

AckerRadar set out to unify these data sources in a single system — **multi-tenant** (multiple farms sharing one platform), **near-real-time**, and accessible via **standardized REST APIs** for frontend clients (e.g., QGIS-based map applications).

My task: design and implement the backend API that performs this aggregation — with a particular focus on plot (land parcel) management, sensor data integration, and geo-spatial queries.

Requirements

The non-functional requirements were strict from the start:

- **Multi-tenant isolation** — one farm's data must never be visible to another, not even through faulty queries.
- **Geo-spatial queries** — "Which sensors are inside this plot?", "Which weather station is closest to my sensor?" must be answered performantly.
- **External API integration** — OpenWeatherMap (weather), MATIF/Euronext (grain prices), ALKIS (official cadastral data). Each source has its own quirks: rate limits, format inconsistencies, occasional outages.
- **API-first design** — complete OpenAPI 3 documentation so frontend teams can develop in parallel.
- **JWT-based authentication** with tenant context propagation through all layers.

Architecture

System Overview

The system follows a classic layered architecture with clear separation between external data sources, the aggregation API, and consuming clients.

External data sources (OpenWeatherMap, MATIF/Euronext, regional grain exchanges) are connected via dedicated fetcher and scraper modules. The central REST API serves aggregated data to authenticated clients — including QGIS maps and IoT field sensors.

Service Structure

Within the API, a coordinating `DataSyncService` orchestrates the domain services. Each domain service encapsulates one data domain and one external data source:

- `WeatherService` — connects to OpenWeatherMap, maps sensors to their nearest weather stations
- `MarketDataService` — aggregates commodity data from multiple exchange sources
- `BlogScraperService` — web scraping for regional grain exchanges whose APIs are not publicly available
- `AuthenticationService` — JWT generation and validation

This separation follows the Single Responsibility Principle and makes each service independently testable and replaceable.

Data Model

The most interesting architectural element is the **atomic data structure**: instead of duplicating status, timestamps, and location as fields in every entity, three reusable base components exist:

- `Distinction` — encapsulates status, category, notes, plus a Timestamp and optionally a Location
- `Timestamp` — isolated time value, separately indexable
- `Location` — geo-coordinates plus an optional GeoJSON Point

Every domain entity (Plot, Sensor, SensorData, MarketData, WeatherData, etc.) references a `Distinction` object instead of maintaining its own status enums and time fields. This yields two concrete benefits:

1. **Consistency** — a status update follows the same path everywhere. No more "I forgot to update the `updatedAt` column" bugs.
2. **Audit capability** — every status change is automatically traceable in time, without a separate audit table.

The trade-off: additional indirection on queries and higher write overhead. In practice, this paid off because audit requirements in the agricultural sector (fertilizer regulation, organic certification) are non-negotiable.

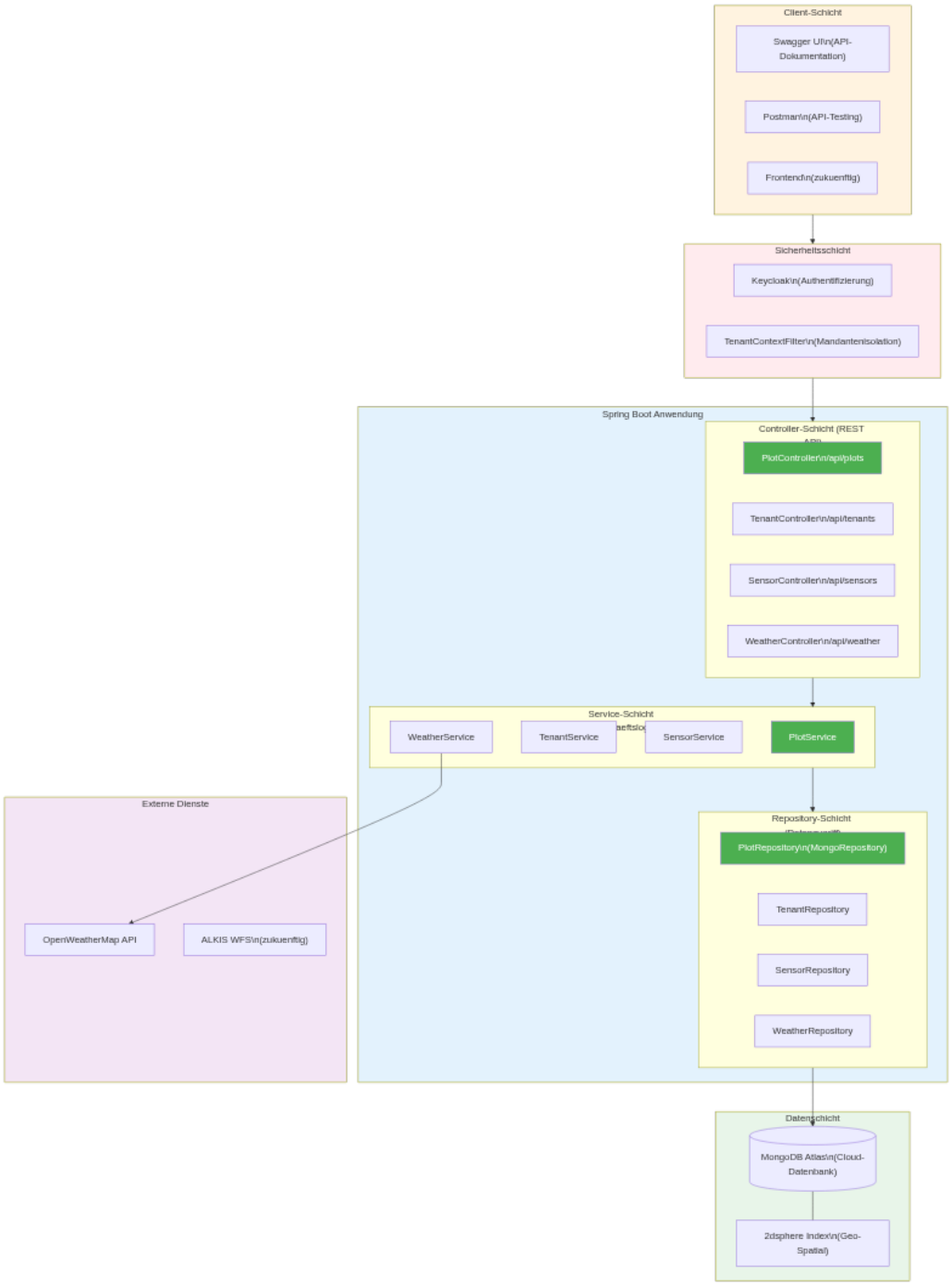

Figure 1: *AckerRadar System Architecture*

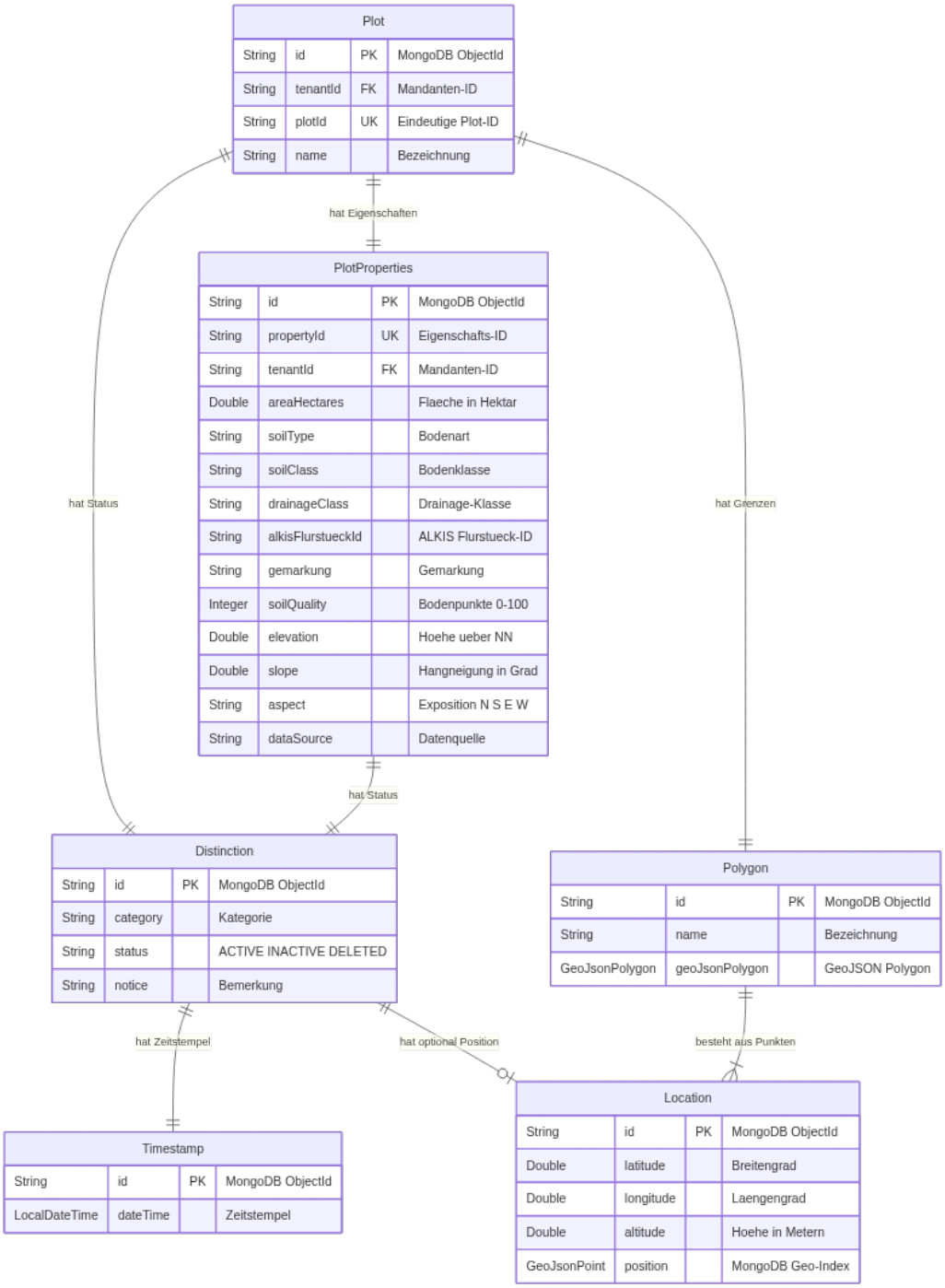

Figure 2: *ER Diagram – Plot Management System*

Plot Management Class Diagram

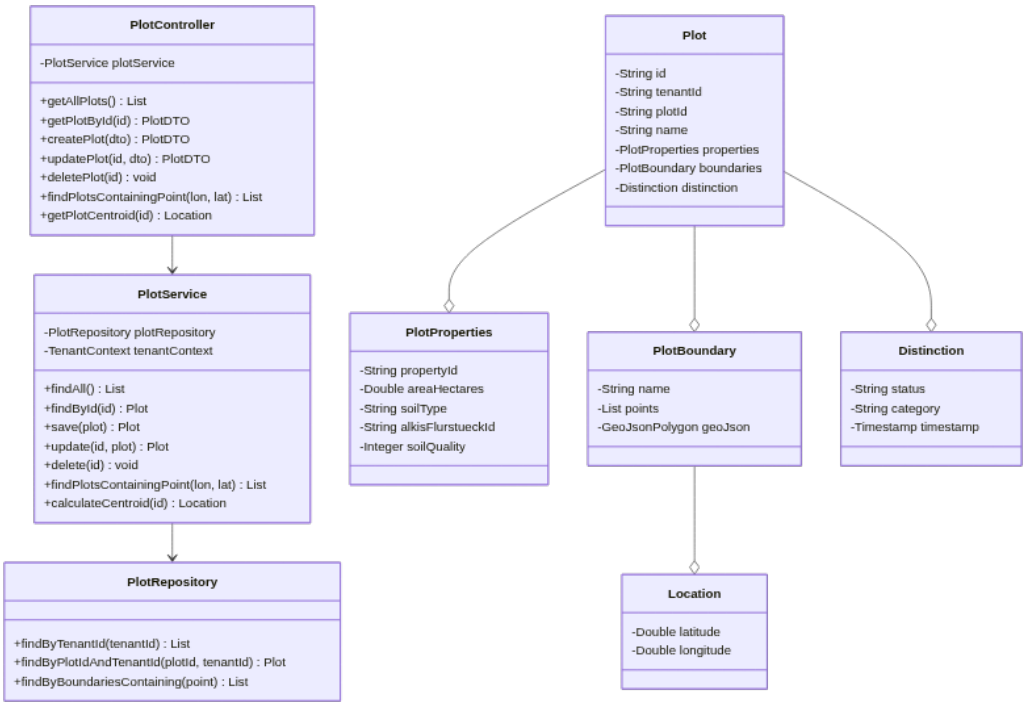

Figure 3: *UML Class Diagram – Plot Management*

A `Plot` (land parcel) consists of several reusable components: `PlotProperties` (soil type, area, ALKIS reference), `Polygon` (geo-boundaries from at least 3 `Location` points), and a `Distinction`. Sensors are assigned to the plot whose polygon contains their GPS coordinates — not through manual linking. This prevents inconsistencies when sensors are physically relocated.

Technical Decisions and Trade-offs

Why MongoDB Instead of PostgreSQL/PostGIS

The original specification called for MySQL. During the project, the team switched to MongoDB for three reasons:

1. **Schema evolution** — the sensor data models changed multiple times during development (from "just temperature and moisture" to "20+ sensor types including NPK values"). MongoDB allows this evolution without migration pain.
2. **Native geo-spatial indexing** — MongoDB supports 2dsphere indexes for GeoJSON out of the box, with operators like `$geoWithin`, `$nearSphere`, and `$geoIntersects`. PostgreSQL would have required PostGIS as an extension to install and maintain.
3. **Document structure fits the domain** — a sensor reading with nested measurements, metadata, and calibration info is more natural as a document than normalized across 5 tables.

The trade-off: no true ACID across multiple collections (not at all before MongoDB 4.0, limited since). For this domain, that was acceptable — sensor data is append-only, and the few operations requiring transactional guarantees (e.g., tenant creation) were deliberately designed as sequential workflows.

Why Spring Boot Instead of Javalin

The specification originally called for Javalin — a lightweight web framework that suited the initially planned scope. As the feature set grew (multi-tenant security, JWT, OpenAPI, repository layer, aggregation pipelines), Spring Boot became the better choice: the ecosystem (Spring Security, Spring Data MongoDB, springdoc-openapi) saved weeks of implementation and maintenance effort. The price:

higher memory footprint and longer startup time — acceptable for a server application that starts once and then runs.

DBRef vs. Manual References

In the MongoDB world, there are two ways to model relationships: `DBRef` (built-in mechanism with automatic resolution) or manual references (plain string IDs). Initially, DBRefs were used because they seem "more automatic." In practice, DBRefs turned out to be difficult to work with in complex aggregation pipelines, and lazy loading didn't work reliably when the tenant context was set via a filter.

The solution: a hybrid approach. Structural relationships (Plot → PlotProperties, Plot → Polygon) remain DBRefs because they are usually loaded together. Tenant references (`tenantId` string in every document) are denormalized instead — this significantly speeds up tenant-filtered queries and simplifies filter logic.

Multi-Tenancy

Multi-tenant isolation was implemented through three mechanisms:

1. **`tenantId` as a mandatory field in every domain document** — denormalized so every query can be filtered efficiently.
2. **`TenantContext` as a ThreadLocal** — per HTTP request, the tenant is extracted from the JWT and stored in a ThreadLocal. Repositories access it automatically.
3. **Compound indexes with `tenantId` in first position** — `{tenantId: 1, createdAt: -1}` instead of just `{createdAt: -1}`. This prevents MongoDB from accidentally scanning across tenants.

The trade-off with ThreadLocals: you must be disciplined about clearing them after every request (memory leaks in the connection pool). This was solved via a `Filter` in the Spring Security chain.

Sequence Diagram: Creating a Plot

The plot creation flow illustrates how the layers interact — including tenant isolation and atomic components:

1. Client sends `POST /api/v1/plots` with a JWT.
2. Spring Security validates the token, extracts the tenant, and sets the `TenantContext`.
3. The controller delegates to `PlotService`.
4. `PlotService` creates a `Distinction` object with the current timestamp.
5. Polygon and PlotProperties are persisted separately (own collections, own IDs).
6. The `Plot` itself references these via DBRef and is saved to the `plots` collection.
7. The response with the fully resolved Plot object goes back to the client.

Challenges and Solutions

GeoJSON Coordinate Order

GeoJSON specifies `[longitude, latitude]` — most humans think in `[latitude, longitude]`. Spring Data MongoDB follows the GeoJSON spec, OpenWeatherMap mixes both conventions, and ALKIS data arrives in yet another format. Three days of debugging a mysterious bug — sensors were being located on a plot in the North Sea instead of Brandenburg — led to a dedicated `CoordinateConverter` utility that centralizes all conversions with unambiguous method names (`fromLatLon`, `fromGeoJson`).

Lesson learned: When a format conflict is subtle, write the converter once centrally and make it impossible to repeat the mistake.

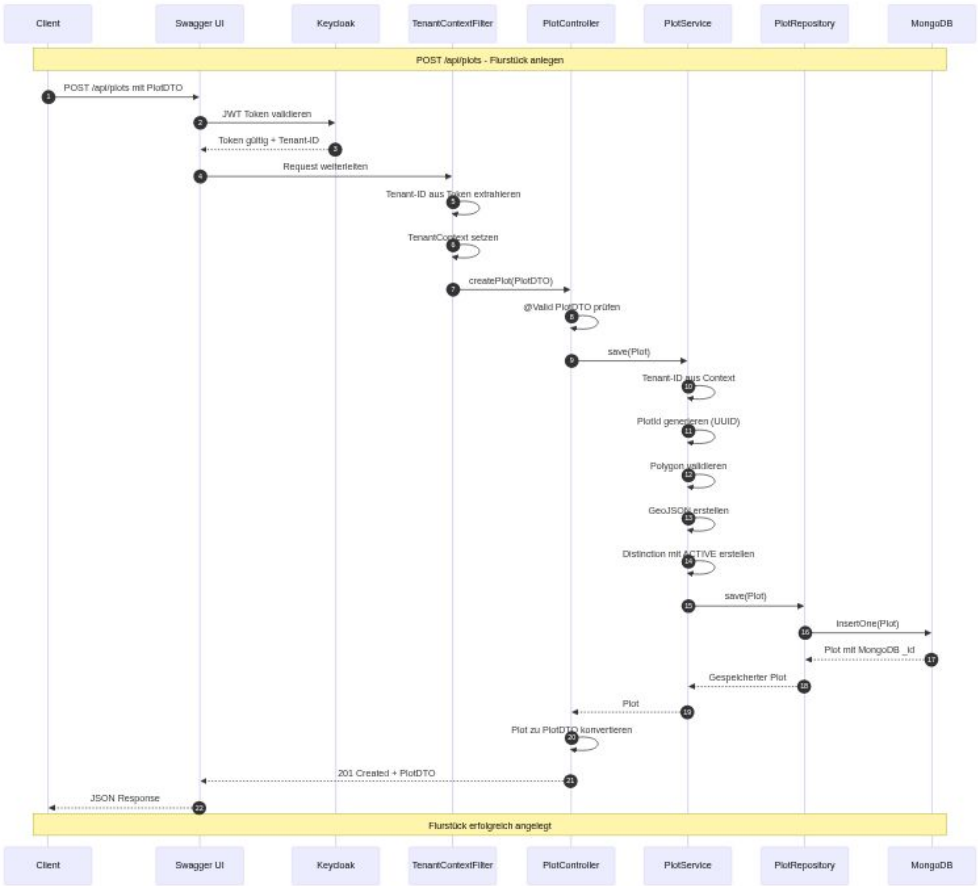

Figure 4: *Sequence Diagram – Plot Creation*

Spring Boot 3 Migration

During the project, Spring Boot 3 was released, bringing the migration from `javax.*` to `jakarta.*`. Several dependencies (particularly older MongoDB driver versions) were not yet compatible. The solution: targeted upgrades of individual dependencies, combined with a thin adapter layer for one critical library that didn't have a Jakarta version yet.

MockMvc + Spring Security

Testability of the security layer was its own challenge. `@WithMockUser` works for many standard cases, but as soon as the tenant context needs to be derived from the JWT, additional test configuration is required. In the IHK documentation, this point was deliberately documented transparently — including the workarounds — rather than maintaining an "all tests green, no problems" facade. This honesty was received positively during the IHK oral examination.

Results

- **Fully functional REST API** with approximately 40 endpoints, documented via OpenAPI 3 (Swagger UI).
- **Multi-tenant capable** with tenant isolation at the database, service, and controller levels.
- **Geo-spatial queries** for sensor-to-plot assignment, nearest weather station lookup, polygon-based plot search.
- **External integrations** for OpenWeatherMap and ALKIS fully functional; MATIF connection prepared as a stub.
- **Passed IHK certification exam** as IT Specialist for Application Development (Fachinformatiker Anwendungsentwicklung).

The code itself is proprietary and not publicly available. This case study documents architecture decisions and lessons learned at a level that works without code disclosure.

Lessons Learned

Atomic data structures are worth their weight when audit requirements matter. The early decision to use `Distinction`/`Timestamp`/`Location` as reusable components prevented inconsistencies throughout the entire project lifecycle. In a project without audit requirements, the overhead would not be justified — here, it was.

MongoDB is an excellent choice for geo-data and evolving schemas, but not a drop-in replacement for relational databases. DBRef limitations, lack of multi-document ACID (before 4.0), and the need to manually optimize aggregation pipelines are real costs. The decision pays off when the domain justifies it.

Honest documentation > polished documentation. In the IHK documentation, technical problems (MockMvc complexity, Spring Boot 3 migration) were described transparently rather than hidden. This worked better in the oral exam than any sugar-coated presentation — examiners recognize the difference between "everything was easy" and "I solved these three problems, here's how."

Multi-tenancy must be designed in from day one. Retrofitting tenant isolation is significantly more expensive than having `tenantId` in every document from the start and structuring compound indexes accordingly.

About Me

Yves Kühn, Berlin. Career changer into software development via IHK vocational retraining as IT Specialist for Application Development (passed 2026). Focus areas: Java backend development, MongoDB, REST API design, increasingly AI integration and AI agent systems. Open-source advocate, maintainer of **Orbis** (own software, GitHub).

Looking for junior or career-changer positions in backend development, remote or Berlin-based.